

LINEAR-SIZE GRAPHS FOR DISJOINT AXIS-ALIGNED RECTANGLES

RUXANDRA TUTUNARU AND ALEXANDRU BENESCU

ABSTRACT. This paper explores the problem of representing the relative horizontal and vertical positions of n pairwise disjoint axis-aligned rectangles in the plane. Motivated by the computation of braids from approximate point positions, we want to reduce the number of relations needed to describe these configurations. While a naive representation requires $O(n^2)$ directed edges to capture all pairwise relations, we prove that all information can be efficiently encoded using only $O(n)$ edges. Furthermore, we present an $O(n \log n)$ algorithm to construct these linear-size graphs.

1. INTRODUCTION

Suppose that we have n pairwise disjoint axis-aligned rectangles in the plane. We want to describe their relative positions using two directed graphs: one for the horizontal direction and one for the vertical direction.

This problem arises from a topological one: computing geometric braids. In practice, we compute braids whenever the position of the points is known approximately, rather than exactly. In this setting, we only have access to basic separation information, such as knowing whether one point is strictly to the left of or strictly below another. Bounding these approximate positions with error intervals replaces the points with two-dimensional boxes, which is where our configuration of disjoint axis-aligned rectangles comes from.

To organize this relative data, we introduce the notion of an *arrangement*. An arrangement is a structure consisting of strict partial orders that record the horizontal and vertical separations between our rectangles. Collecting these relations into an arrangement makes it easy to recover the combinatorial braid.

A direct approach to building this arrangement is to compare every rectangle with every other rectangle. However, this is a quadratic, inefficient approach that creates a dense graph with $O(n^2)$ relations. Since relations of the same type can be combined transitively along a directed path, there is some hope of removing many of the redundant edges. However, relations of different types (horizontal versus vertical) cannot be combined in this way, so it is not immediately clear that a small, optimized representation exists.

Our main result shows that it does. Given n pairwise disjoint axis-aligned rectangles, we construct two directed graphs with $O(n)$ edges which still contain enough information to compare every pair of rectangles in at least one of the two directions. We also give an $O(n \log n)$ algorithm for constructing these graphs.

In what follows, we will prove these theorems:

Theorem 1. *Let $B_1, \dots, B_n$ be pairwise disjoint axis-aligned rectangles in the plane. There exist two directed graphs G_{re} and G_{im} on the node set $[1, n]$ such that*

- (i) *if $i \rightarrow j$ is an edge of G_{re} , every point of B_i has x -coordinate smaller than every point of B_j ;*

- (ii) if $i \rightarrow j$ is an edge of G_{im} , every point of B_i has y -coordinate smaller than every point of B_j ;
- (iii) for any two distinct nodes there is a directed path from one to the other in at least one of the two graphs;
- (iv) G_{re} and G_{im} have $O(n)$ edges in total.

Theorem 2. *The two graphs of Theorem 1 can be computed in $O(n \log n)$ time.*

Throughout, $B_1, B_2, \dots, B_n$ are pairwise disjoint axis-aligned rectangles in the plane, and G_{re} and G_{im} are the two directed graphs constructed in Section 3, on the node set $[1, n]$. We identify a node with its rectangle whenever no confusion arises.

2. REDUCTION TO (ALMOST) A TILING

The construction is easier to achieve when the rectangles cover the plane exactly. Although this is not always possible, we can still enlarge them so that the uncovered regions of the plane are easy to characterize.

Let us expand the rectangles as follows. We first deal with the upward expansion. For every rectangle from 1 to n (in this order), we expand it by translating its top side upwards while the interiors are still disjoint. Note that some of the expanded rectangles may be infinite but this does not affect us in any way. We repeat the process for the other three directions. Now, we have obtained a new set of rectangles $A_1, A_2, \dots, A_n$ such that B_k is included in A_k for all $k \in [1, n]$.

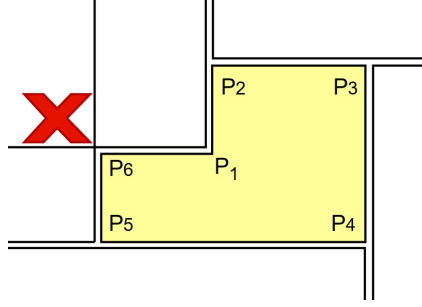
Claim: If there exist graphs G_{re} and G_{im} satisfying the four conditions in the statement for rectangles $A_1, A_2, \dots, A_n$ rather than $B_1, B_2, \dots, B_n$, then the same graphs are satisfying the conditions for rectangles $B_1, B_2, \dots, B_n$.

Proof: Let $i \rightarrow j$ be an edge in one of the two graphs. Any point inside rectangle A_i has the x/y -coordinate less than the x/y -coordinate of any point inside rectangle A_j . Since the interior of the rectangle B_i is included in the interior of the rectangle A_i and the interior of the rectangle B_j is included in the interior of the rectangle A_j , then any point inside rectangle B_i has the x/y -coordinate less than the x/y -coordinate of any point inside rectangle B_j . Because the two graphs remain unchanged when transitioning from one collection of rectangles to the other one, the last two conditions in the statement are satisfied as well.

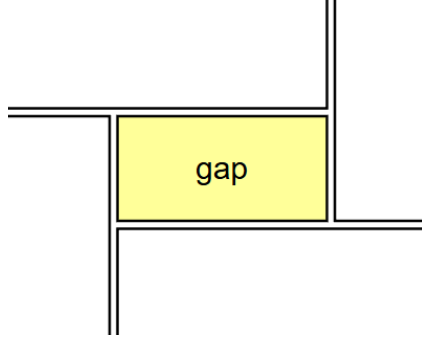
Let us now characterize the regions of the plane that are not covered by any A_i , $i \in [1, n]$. We call such regions gaps. Let the polygon $P_1 P_2 \dots P_k$ be a gap.

Claim: $P_1 P_2 \dots P_k$ is convex.

Proof: Assume by contradiction that $P_1 P_2 \dots P_k$ is concave. Since all angles of the polygon are either of 90° or of 270° , we can say without loss of generality that $\angle P_1 = 270^\circ$. Since no side of any rectangle can be included in any side of the gap, there exists a rectangle with a corner in P_1 , with one side exceeding $P_1 P_2$ and with another side exceeding $P_1 P_k$. Inductively, for every $i \in [2, k-1]$, there exists a rectangle with a corner in P_i and with a side exceeding $P_i P_{i+1}$. This will determine two rectangles to overlap next to the point P_k , leading to a contradiction.



Thus, $P_1 P_2 \dots P_k$ is convex. Specifically, it is a rectangle. So, any gap is surrounded by four rectangles, the only possible configuration being the one below (along with its reflection). (*)



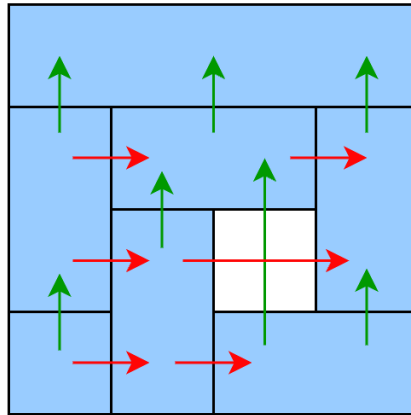
Since each corner of each rectangle has at most one associated gap, the number of gaps is at most $4n$, so linear in n .

For simplicity, we will now call the rectangles back as $B_1, B_2, \dots, B_n$ instead of $A_1, A_2, \dots, A_n$.

3. THE CONSTRUCTION

Two rectangles are *adjacent* if their boundaries meet in a segment of positive length, or if they are both adjacent to the same gap G without their boundaries meeting. We say that Y is *immediately to the right of* X if the right side of X and the left side of Y meet in a segment of positive length, and symmetrically for *immediately to the left of*, *immediately above* and *immediately below*; every pair of rectangles whose boundaries meet stands in exactly one of these four relations.

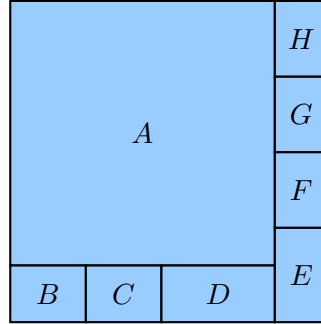
For every $(i, j) \in [1, n]^2, i \neq j$, let $i \rightarrow j$ be an edge in G_{re} or G_{im} (depending on their relative positions) if rectangles B_i and B_j are adjacent. Since the resulting graphs are planar, their number of edges is linear in n . These are the standard edges, but they are not enough.



By (*) each side of a gap has exactly one rectangle glued to it. Let us say that Y is *directly above* X if Y is immediately above X , or if there is a gap whose bottom side is glued to X and whose top side is glued to Y , and let us define *directly below*, *directly to the left of* and *directly to the right of* in the same way. This is just adjacency, recorded with a direction.

Let s be a side of a rectangle X and let ℓ be the line containing s . A rectangle Y *hangs over* s if Y has a point strictly beyond ℓ , on the side of ℓ not containing X . We use the preposition suggested by the side in question and say, for the four sides in turn, that Y hangs over the left side, hangs over the right side, hangs above the top side or hangs below the bottom side of X ; in coordinates these read $x_{\min}(Y) < x_{\min}(X)$, $x_{\max}(Y) > x_{\max}(X)$, $y_{\max}(Y) > y_{\max}(X)$ and $y_{\min}(Y) < y_{\min}(X)$. When Y hangs over the left side of X we also say that Y *extends to the left of* X .

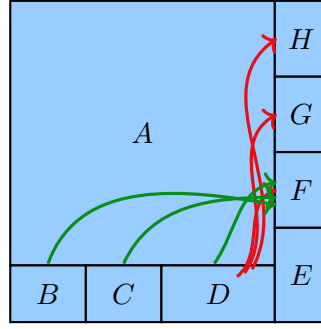
Consider for instance the following arrangement.



In this arrangement B is to the left of H and also below it, so we must be able to compare them: there has to be a path from B to H in one of the two graphs. The standard edges do not give one. Starting from B in G_{re} we can only walk right along the bottom row, to C , then to D , then to E , and there is nothing to the right of E . Starting from B in G_{im} we can only walk up to A , and there is nothing above A . To get to H we would have to go right along the bottom row and then up along the right column, but the first part uses edges of G_{re} and the second part edges of G_{im} , while a path must stay inside a single graph. So what is missing is an edge that turns the corner, and this is what the around-the-corner edges provide.

Now we add the around-the-corner edges. For each rectangle B_k we look, in turn, at each of its four corners; we describe the construction at the bottom-right corner, the other three being symmetric.

Let B_i be the rightmost rectangle among those directly below B_k that do not hang over the right side of B_k , and let B_j be the lowest rectangle among those directly to the right of B_k that do not hang below the bottom side of B_k . For every rectangle B_t directly to the right of B_k that does not hang below the bottom side of B_k , we add the edge $i \rightarrow t$ to G_{re} . For every rectangle B_t directly below B_k that does not hang over the right side of B_k , we add the edge $t \rightarrow j$ to G_{im} . We repeat this procedure for the other three corners of B_k .



For instance, in the figure above $B_k = A$, with B, C, D directly below it and E, F, G, H directly to its right. Since E hangs below the bottom side of A while F, G, H do not, and since D is the rightmost rectangle below A that does not hang over its right side, we obtain $i = D$ and $j = F$, and we add the edges $D \rightarrow F$, $D \rightarrow G$, $D \rightarrow H$ to G_{re} and $B \rightarrow F$, $C \rightarrow F$, $D \rightarrow F$ to G_{im} (drawn in red and green, respectively). Every added edge points from a rectangle with smaller coordinate to one with larger coordinate, since D lies strictly to the left of F, G, H and B, C, D lie strictly below F ; a gap between A and one of its neighbors only pushes the two apart, so the first two conditions in the statement continue to hold. Note that $B \rightarrow F$ is one of the edges we just added, so now $B \rightarrow F \rightarrow G \rightarrow H$ is a path from B to H in G_{im} : the pair that the standard edges could not compare is taken care of by the corner rule.

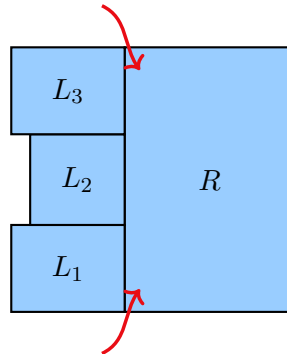
Claim: The number of around-the-corner edges is linear in n .

Proof: We associate each of the around-the-corner edges to a side of a rectangle so that every side gets at most two; as there are $4n$ sides, this leaves at most $8n$ such edges.

At the bottom-right corner of B_k we associate each real edge $i \rightarrow t$ to the left side of t and each imaginary edge $t \rightarrow j$ to the top side of t ; the other three corners are symmetric.

The rectangles directly to the left of a rectangle R are still stacked in order from bottom to top: the left side of R is cut into consecutive pieces, each of them shared with a rectangle immediately to the left of R or a side of a gap, and each gap contributes exactly one rectangle.

Now consider the left side of R and the rectangles stacked directly to its left. An around-the-corner edge into R is created only at the bottom-right corner of the lowest of these neighbors or at the top-right corner of the highest one: any neighbor in between is straddled by R both above and below, so neither of its right corners reaches R (see the figure). A gap between two consecutive neighbor changes nothing, since the lower of the two fills, together with that gap, the strip immediately to the left of R below the higher one. Hence the left side of R receives at most two edges, and by symmetry so does every side. The claim follows.

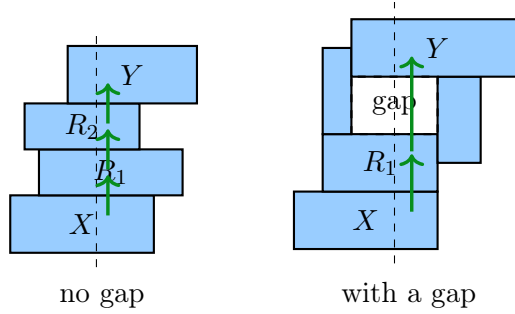


4. COMPARABILITY

Let us write $X \Rightarrow_{re} Y$ (respectively $X \Rightarrow_{im} Y$) for a directed path from X to Y in G_{re} (respectively in G_{im}), and let us call re and im the two *colors*. Given a third rectangle A , we say that A *reaches* X and Y in the same color if there is a color c with $A \Rightarrow_c X$ and $A \Rightarrow_c Y$.

Lemma 1. *Let X and Y be rectangles. If the projections of X and Y onto the x -axis overlap and X lies below Y , then $X \Rightarrow_{im} Y$. Symmetrically, if the projections onto the y -axis overlap and X lies to the left of Y , then $X \Rightarrow_{re} Y$.*

Proof: A vertical line through the interior of the overlap, chosen to avoid every rectangle corner, crosses a sequence of rectangles $X = R_0, R_1, \dots, R_p = Y$ from bottom to top; the rectangles and the gaps cover the plane by Section 2, so the sequence is well defined. Each consecutive pair shares a horizontal segment of positive length, hence $R_\ell \rightarrow R_{\ell+1}$ is a standard edge of G_{im} . If a gap lies between them the edge is still there: the line enters the gap through its bottom side and leaves through its top side, so R_ℓ and $R_{\ell+1}$ are adjacent. The concatenation of these edges is a path from X to Y in G_{im} . Reflecting in the diagonal turns the picture into the horizontal one and turns G_{im} into G_{re} , so the same argument with a horizontal line gives a path from X to Y in G_{re} .



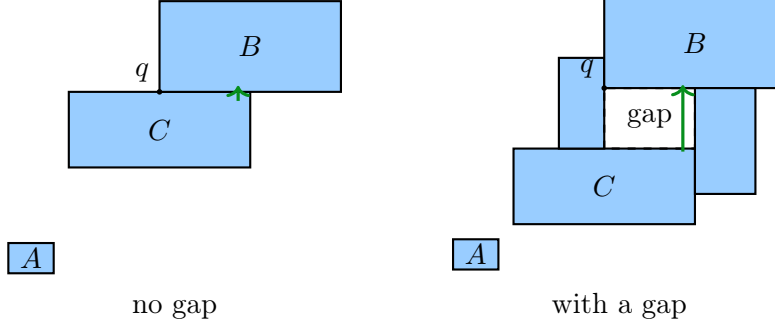
The path produced by Lemma 1 has a prescribed color, determined by the axis on which the two projections overlap. The next lemma is what allows a path to be extended through a corner, where the color would otherwise have to change.

Let us say that B lies to the *top right* of A if the projections of A and B onto the x -axis have disjoint interiors, with that of A to the left, and the projections onto the y -axis have disjoint interiors as well, with that of A below. This is weaker than asking the projections to be disjoint: they are allowed to share an endpoint, so that a single horizontal line may carry the top side of A and the bottom side of B . That borderline configuration is included on purpose, because Lemma 1 says nothing about it and it therefore has to be covered here.

The proof of the next lemma is an induction on a strictly ordered set, which we recall. Let S be a set, let $<$ be a strict order on S , that is, a relation which is irreflexive and transitive, and suppose that $<$ is well founded. Let P be a property of the elements of S . If for every $x \in S$ the assumption that $P(y)$ holds for all $y < x$ implies that $P(x)$ holds, then $P(x)$ holds for every $x \in S$. In practice one fixes $x \in S$, assumes $P(y)$ for every $y < x$, and proves $P(x)$. No base case has to be treated separately: if x is minimal for $<$ then there is no $y < x$ at all, so the assumption gives nothing and $P(x)$ has to be proved on its own, which is exactly what a base case would ask for.

Lemma 2. *Let A be a rectangle, let B be a rectangle lying to the top right of A , let q be the bottom-left corner of B , and let C be a rectangle adjacent to B at q . Then A reaches B and C in the same color.*

Two rectangles are adjacent to B at q , the one below B and the one to its left, and the statement covers both. Without gaps, the rectangle below B at q is the one whose top side contains q . With gaps there is a second possibility: a gap sits at q , and the rectangle below B is then the one below that gap. The two situations are the same for our purposes, since in both of them that rectangle extends to the left of B and is joined to B by a standard edge of G_{im} .



Proof: We use the induction scheme recalled above. The rectangle A is fixed throughout; write p for its top-right corner. Let S be the set of pairs (B, C) as in the statement, that is, of pairs in which B lies to the top right of A and C is adjacent to B at its bottom-left corner. Let $P(B, C)$ be the property that A reaches B and C in the same color, and order S by declaring

$$(B, C) < (B', C') \quad \text{when} \quad \|p - q\| < \|p - q'\|,$$

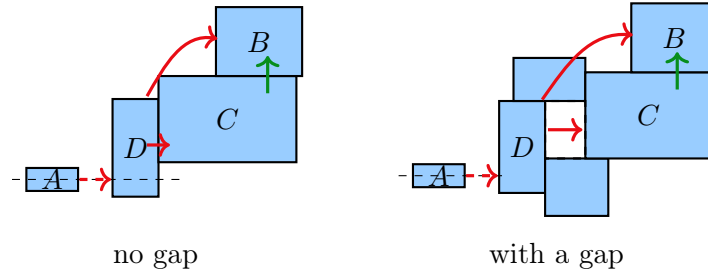
where q and q' are the bottom-left corners of B and of B' . This relation is irreflexive and transitive, being inherited from the order on the real numbers, and it is well founded because there are finitely many rectangles and hence finitely many pairs. So we may fix $(B, C) \in S$, assume $P(B', C')$ for every $(B', C') < (B, C)$, and prove $P(B, C)$.

Reflecting in the diagonal exchanges G_{re} and G_{im} and exchanges the two rectangles adjacent to B at q , so we may assume that C is the one below B . Then C extends to the left of B and $C \rightarrow B$ is an edge of G_{im} , so B never hangs over the left side of C .

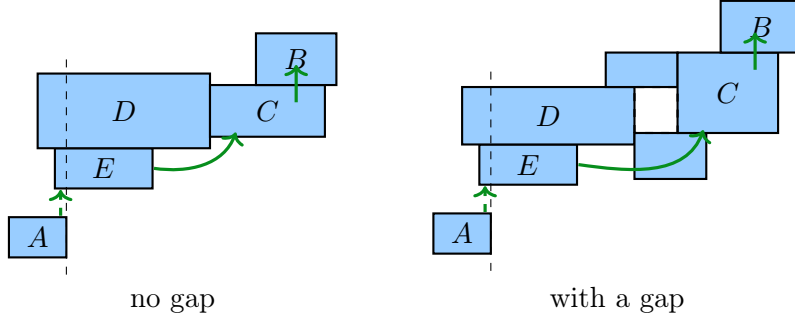
If $A \Rightarrow_{im} C$, composing with $C \rightarrow B$ gives $A \Rightarrow_{im} B$ and the color im works. We may thus assume that there is no path from A to C in G_{im} . (★★)

Since the statement assumes nothing about the position of C relative to A , we may distinguish cases according to C itself rather than according to one of its neighbors. Three cases arise, and only the last one appeals to the induction hypothesis.

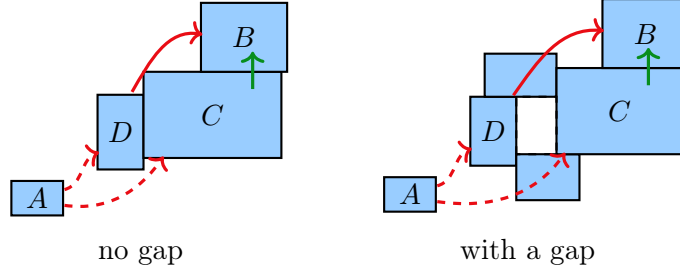
Case 1: the projections of A and C onto the y -axis overlap. As A lies to the left of C , Lemma 1 gives $A \Rightarrow_{re} C$. The horizontal line used in that lemma enters C through its left side, so the last edge of the path comes from a rectangle D directly to the left of C , and $A \Rightarrow_{re} D$ as well.



Case 2: the projections of A and C onto the x -axis overlap. Then A lies below C , so Lemma 1 gives $A \Rightarrow_{im} C$ at once, contradicting (★★); this case does not occur.

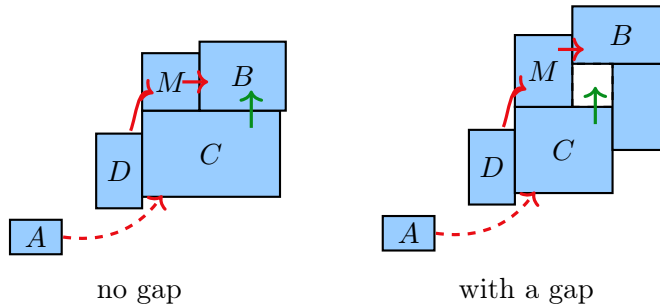


Case 3: the projections of A and C are disjoint on both axes. Then C itself lies to the top right of A . Let D be the rectangle adjacent to C at its bottom-left corner from the left; such a rectangle exists because the expanded rectangles and the gaps cover the plane, by Section 2. The pair (C, D) therefore belongs to S , and the bottom-left corner of C is closer to p than q is, since C lies below B and extends to the left of B . Hence $(C, D) < (B, C)$, and the induction hypothesis gives that A reaches C and D in the same color c . If $c = im$ then $A \Rightarrow_{im} C$ contradicts $(\star\star)$, so $c = re$; that is, $A \Rightarrow_{re} D$ and $A \Rightarrow_{re} C$.



Reaching B . Case 2 does not occur, and Cases 1 and 3 both produce a rectangle D directly to the left of C with $A \Rightarrow_{re} D$ and $A \Rightarrow_{re} C$. By $(\star\star)$ the color im is unavailable for C , so it remains to produce a path $A \Rightarrow_{re} B$.

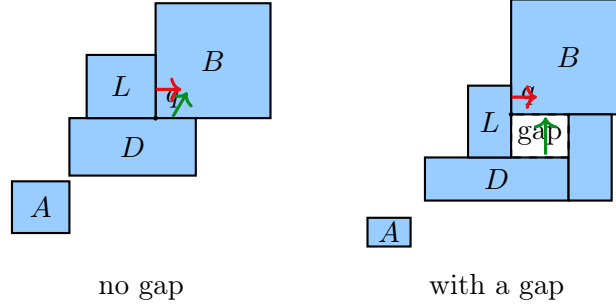
Let M be the leftmost rectangle directly above C that does not hang over the left side of C . The around-the-corner construction at the top-left corner of C adds the edge $D \rightarrow M$ to G_{re} , since D is directly to the left of C without hanging above its top side. If D does hang above the top side of C , that rule does not apply, but D then touches C and fills the strip immediately to its left, so its projection onto the y -axis meets that of M and Lemma 1 gives $D \Rightarrow_{re} M$. Because C extends to the left of B , the rectangle B does not hang over the left side of C and is therefore one of the candidates from which M was chosen, so M lies weakly to the left of B . Both are directly above C , and two rectangles on the same side of C are joined by a standard edge, so $M \Rightarrow_{re} B$. Hence $A \Rightarrow_{re} D \rightarrow M \Rightarrow_{re} B$, and the color re works. This completes the induction.



Claim: For any two distinct rectangles there is a directed path from one to the other in G_{re} or in G_{im} .

Proof: Two distinct rectangles are disjoint, so their projections are disjoint on at least one axis. If they overlap on the other axis, Lemma 1 already connects them. We may therefore assume the two rectangles are separated on both axes; up to a reflection, one of them, A , lies strictly to the lower-left of the other, B .

Consider the bottom-left corner q of B , and let D and L be the rectangles at q below B and to its left, so that $D \rightarrow B$ is an edge of G_{im} and $L \rightarrow B$ is an edge of G_{re} . Up to a reflection, D reaches across q , so that D extends to the left of B .



The triple (A, B, D) is then exactly the configuration of Lemma 2, which gives a directed path from A to B in G_{re} or in G_{im} . If A shares a projection with D the lemma does not apply, but A then lies below D or to its left. In the first case Lemma 1 gives $A \Rightarrow_{im} D \rightarrow B$; in the second it gives a path $A \Rightarrow_{re} D$ whose last step comes from a rectangle directly to the left of D , and the argument of the previous paragraph, applied to D in place of C , carries that rectangle on to B . This proves the claim and, with it, that the two graphs satisfy all the conditions in the statement.

5. COMPUTING THE GRAPHS

Given n and the rectangles $B_1, B_2, \dots, B_n$ (as two pairs of floating-point numbers denoting the bottom-left and top-right corners of each rectangle), we present an efficient algorithm for determining the graphs G_{re} and G_{im} .

5.1. Coordinates normalization. For each of the two axis independently, we normalize the coordinates of the rectangles by sorting them in ascending order and renumbering them from 1 to $2n$. (Note that we do not allow equal coordinates. We randomly differentiate between two equal coordinates.)

5.2. Rectangles expansion. For each of the four directions, we want to determine for every rectangle which other rectangle it will hit first if we expand it in that direction. We will describe the procedure for the upward expansion. The other three directions can be processed analogously. For the upward expansion, given the top side of a rectangle, we need to find the bottom side of the first rectangle it would intersect if translated upwards. From now on, we will refer to a horizontal side by its two x -coordinates, a and b (with $a < b$).

The Brute-Force Approach ($O(n^2)$). The most intuitive way to solve this is to check every pair of rectangles. For a given rectangle B_i with its top side at y -coordinate $y_{top}^{(i)}$ and x -interval $[a_i, b_i]$, we can iterate through all other rectangles B_j ($j \neq i$). We consider B_j as a candidate to be hit if:

- (1) The bottom side of B_j , located at $y_{bot}^{(j)}$, is strictly above the top side of B_i (i.e., $y_{bot}^{(j)} \geq y_{top}^{(i)}$).
- (2) Their projections onto the x -axis overlap (i.e., $[a_i, b_i] \cap [a_j, b_j] \neq \emptyset$).

Among all such valid candidate rectangles B_j , the one that B_i will hit first is the one that minimizes $y_{bot}^{(j)}$. We record this minimum and add the corresponding edge to our graph.

Since finding the target for one rectangle requires scanning up to $n - 1$ other rectangles, the overall time complexity for expanding all n rectangles in one direction is $O(n^2)$. This approach is too slow to achieve the desired efficiency for large n .

The Linearithmic Approach ($O(n \log n)$). To achieve an efficient $O(n \log n)$ complexity, we can use a sweep-line algorithm paired with segment trees.

We process all $2n$ horizontal sides (both top and bottom sides of all rectangles) in descending order of their y -coordinates. When our sweep-line reaches the top side of a rectangle B_i , any rectangle B_j that is above B_i has already had its bottom side processed. Thus, we just need to query the processed bottom sides to find the one with the minimum y -coordinate that overlaps with $[a_i, b_i]$.

Two intervals $[a_i, b_i]$ and $[a_j, b_j]$ overlap if and only if at least one of the following conditions is true:

- $a_j \in [a_i, b_i]$ (the left endpoint of B_j falls inside the interval of B_i)
- $a_i \in [a_j, b_j]$ (the left endpoint of B_i falls inside the interval of B_j)

Because there are two distinct ways for the intervals to intersect relative to their left endpoints, a single standard segment tree is insufficient. We use two segment trees operating over the normalized x -coordinates. Both trees store data as pairs: (**y-coordinate**, **rectangle_index**), and use the **y-coordinate** to evaluate the minimum during queries.

1. Standard Segment Tree (point-update, range-minimum query):

This tree handles the case where $a_j \in [a_i, b_i]$.

- *Update:* If the current segment $[a, b]$ represents the bottom side of some rectangle B_k , we update the segment tree at point a with the y -coordinate of B_k (and its index k).
- *Query:* If the current segment $[a, b]$ represents the top side of some rectangle B_i , we query the range $[a, b]$ to find the index j of the rectangle with the minimum y -coordinate among those stored in this range.

2. Dual Segment Tree (range-update, point-minimum query):

This tree handles the case where $a_i \in [a_j, b_j]$.

- *Update:* If the current segment $[a, b]$ represents the bottom side of some rectangle B_k , we update the segment tree on the entire range $[a, b]$ with the y -coordinate of B_k (and its index k).
- *Query:* If the current segment $[a, b]$ represents the top side of some rectangle B_i , we query the point a to find the index j of the rectangle with the minimum y -coordinate covering this specific point.

By taking the minimum of the results from the two segment trees, we can correctly determine the first rectangle hit during the upward expansion in $O(\log n)$ time per event. Processing all $2n$ events yields an overall $O(n \log n)$ time complexity per direction.

5.3. Solving the gaps. To resolve the gaps and add the edges between rectangles adjacent across a gap, we need to identify all sets of four rectangles (i, j, k, l) that perfectly enclose an empty region. We can find these configurations efficiently by tracing the boundaries of potential

gaps. Iterating through each rectangle i , we attempt to “walk” around a potential gap in the clockwise direction:

- (1) Let j be the lowest rectangle immediately to the right of i .
- (2) Let k be the leftmost rectangle immediately below j .
- (3) Let l be the highest rectangle immediately to the left of k .
- (4) Finally, verify that i is indeed the rightmost rectangle immediately above l .

If we successfully close this loop and return to i , we then check that no extra edges exist between these four rectangles in either graph. (Checking for edge existence takes constant time.)

Analogously, we search for symmetric gaps by tracing in the other direction.

Whenever we find a valid gap configuration (i, j, k, l) , we cross the gap by adding the edge $l \rightarrow j$ to G_{re} and the edge $k \rightarrow i$ to G_{im} .

5.4. Adding the around-the-corner edges. The final step is to add the around-the-corner edges. We iterate through every rectangle i and examine each of its four corners. Let us use the bottom-right corner as an example.

First, we want to create horizontal relations around the corner. We identify j , the rightmost rectangle immediately below i that does not extend past the right edge of i . Once j is found, we add an edge $j \rightarrow t$ in G_{re} for every rectangle t immediately to the right of i that does not extend below i ’s bottom edge.

Next, we create vertical relations around the same corner. We identify k , the lowest rectangle immediately to the right of i that does not extend below the bottom edge of i . We then add an edge $t \rightarrow k$ in G_{im} for every rectangle t immediately below i that does not extend past i ’s right edge.

We repeat this process for the top-right, bottom-left and top-left corners of every rectangle.

Each of the four main steps of our algorithm runs in $O(n \log n)$ time and requires $O(n)$ space. Thus, the overall complexity of the algorithm is:

- **Time:** $O(n \log n)$
- **Space:** $O(n)$

ACKNOWLEDGMENTS

This work was carried out during an internship at Inria Paris-Saclay from June to September 2026. We thank Pierre Lairez and Alexandre Guillemot for proposing the problem and for their guidance throughout.

REFERENCES

- [1] A. Guillemot and P. Lairez. *Computing braids from approximate data*. arXiv:2601.23073, 2026.

ÉCOLE POLYTECHNIQUE, PALAISEAU, 91120, FRANCE
 Email address: ruxandra-ioana.tutunaru@polytechnique.edu

ÉCOLE POLYTECHNIQUE, PALAISEAU, 91120, FRANCE
 Email address: alexandru-andrei.benescu@polytechnique.edu

APPENDIX A. PYTHON IMPLEMENTATION

The following Python program takes as input a positive integer n (the number of rectangles) and on each of the following n lines, four floating-point numbers separated by spaces (the coordinates of the bottom-left and top-right corners of each rectangle). The program outputs the edges of each of the two graphs (G_{re}, G_{im}) as pairs of integers.

```
def solve_rectangles(n, rectangles):
    """
    n: integer, number of rectangles
    rectangles: list of tuples (x_min, y_min, x_max, y_max)
    returns: (Gre, Gim) as lists of directed edges (u, v)
    """

    def best_min(a, b):
        return a if a[0] < b[0] else b

    def best_max(a, b):
        return a if a[0] > b[0] else b

    # --- Standard Segment Tree: point-update, range-query ---
    def update_st1(tree, node, l, r, pos, val_pair, best_fn):
        if l == r:
            tree[node] = best_fn(tree[node], val_pair)
            return
        mid = (l + r) // 2
        if pos <= mid:
            update_st1(tree, 2 * node, l, mid, pos, val_pair, best_fn)
        else:
            update_st1(tree, 2 * node + 1, mid + 1, r, pos, val_pair,
                        best_fn)
        tree[node] = best_fn(tree[2 * node], tree[2 * node + 1])

    def query_st1(tree, node, l, r, ql, qr, best_fn, null_val):
        if ql <= l and r <= qr:
            return tree[node]
        mid = (l + r) // 2
        res = null_val
        if ql <= mid:
            res = best_fn(res, query_st1(
                tree, 2 * node, l, mid, ql, qr, best_fn, null_val
            ))
        if qr > mid:
            res = best_fn(res, query_st1(
                tree, 2 * node + 1, mid + 1, r, ql, qr, best_fn, null_val
            ))
        return res

    # --- Dual Segment Tree: range-update, point-query ---
    def update_st2(tree, node, l, r, ql, qr, val_pair, best_fn):
        if ql <= l and r <= qr:
            tree[node] = best_fn(tree[node], val_pair)
            return
        mid = (l + r) // 2
```

```

if ql <= mid:
    update_st2(tree, 2 * node, l, mid, ql, qr, val_pair, best_fn)
if qr > mid:
    update_st2(tree, 2 * node + 1, mid + 1, r, ql, qr, val_pair,
                best_fn)

def query_st2(tree, node, l, r, pos, best_fn, null_val):
    res = tree[node]
    if l == r:
        return res
    mid = (l + r) // 2
    if pos <= mid:
        res = best_fn(res, query_st2(
            tree, 2 * node, l, mid, pos, best_fn, null_val
        ))
    else:
        res = best_fn(res, query_st2(
            tree, 2 * node + 1, mid + 1, r, pos, best_fn, null_val
        ))
    return res

def sweep(n, rectangles, direction):
    hit = [-1] * n
    events = []

    if direction == 'UP':
        for i, (x1, y1, x2, y2) in enumerate(rectangles):
            events.append((y1, 0, x1, x2, y1, i))
            events.append((y2, 1, x1, x2, 0, i))
        events.sort(key=lambda e: (-e[0], e[1]))
        best_fn = best_min; null_val = (float('inf'), -1)

    elif direction == 'DOWN':
        for i, (x1, y1, x2, y2) in enumerate(rectangles):
            events.append((y2, 0, x1, x2, y2, i))
            events.append((y1, 1, x1, x2, 0, i))
        events.sort(key=lambda e: (e[0], e[1]))
        best_fn = best_max; null_val = (float('-inf'), -1)

    elif direction == 'RIGHT':
        for i, (x1, y1, x2, y2) in enumerate(rectangles):
            events.append((x1, 0, y1, y2, x1, i))
            events.append((x2, 1, y1, y2, 0, i))
        events.sort(key=lambda e: (-e[0], e[1]))
        best_fn = best_min; null_val = (float('inf'), -1)

    elif direction == 'LEFT':
        for i, (x1, y1, x2, y2) in enumerate(rectangles):
            events.append((x2, 0, y1, y2, x2, i))
            events.append((x1, 1, y1, y2, 0, i))
        events.sort(key=lambda e: (e[0], e[1]))
        best_fn = best_max; null_val = (float('-inf'), -1)

```

```

cross_coords = sorted(
    list(set([e[2] for e in events] + [e[3] for e in events]))
)
cross_map = {val: rank + 1 for rank, val in enumerate(cross_coords)}
M = len(cross_coords)

tree1 = [null_val] * (4 * M + 4)
tree2 = [null_val] * (4 * M + 4)

for e in events:
    _, typ, c_min, c_max, val, i = e
    l, r = cross_map[c_min], cross_map[c_max]
    if typ == 0:
        update_st1(tree1, 1, 1, M, l, (val, i), best_fn)
        update_st2(tree2, 1, 1, M, l, r, (val, i), best_fn)
    else:
        res1 = query_st1(tree1, 1, 1, M, l, r, best_fn, null_val)
        res2 = query_st2(tree2, 1, 1, M, l, best_fn, null_val)
        best_res = best_fn(res1, res2)
        if best_res[1] != -1:
            hit[i] = best_res[1]

return hit

# --- Step 2: Rectangles Expansion ---
hit_up = sweep(n, rectangles, 'UP')
hit_down = sweep(n, rectangles, 'DOWN')

expanded_rectangles = []
for i in range(n):
    x1, y1, x2, y2 = rectangles[i]
    ny1 = rectangles[hit_down[i]][3] if hit_down[i] != -1 else float('-inf')
    ny2 = rectangles[hit_up[i]][1] if hit_up[i] != -1 else float('inf')
    expanded_rectangles.append((x1, ny1, x2, ny2))

hit_right = sweep(n, expanded_rectangles, 'RIGHT')
hit_left = sweep(n, expanded_rectangles, 'LEFT')

Gre_edges = set()
Gim_edges = set()
for i in range(n):
    if hit_right[i] != -1:
        Gre_edges.add((i, hit_right[i]))
    if hit_left[i] != -1:
        Gre_edges.add((hit_left[i], i))
    if hit_up[i] != -1:
        Gim_edges.add((i, hit_up[i]))
    if hit_down[i] != -1:
        Gim_edges.add((hit_down[i], i))

adj_re_out = [[] for _ in range(n)]
adj_re_in = [[] for _ in range(n)]
adj_im_out = [[] for _ in range(n)]

```

```

adj_im_in = [[] for _ in range(n)]

for u, v in Gre_edges:
    adj_re_out[u].append(v)
    adj_re_in[v].append(u)
for u, v in Gim_edges:
    adj_im_out[u].append(v)
    adj_im_in[v].append(u)

def sort_adj(adj, key_idx, reverse):
    return [
        sorted(adj[i], key=lambda x: rectangles[x][key_idx],
               reverse=reverse)
        for i in range(n)
    ]

re_out_min_y2 = sort_adj(adj_re_out, 3, False)
re_in_max_y1 = sort_adj(adj_re_in, 1, True)
re_out_max_y1 = sort_adj(adj_re_out, 1, True)
re_in_min_y2 = sort_adj(adj_re_in, 3, False)

im_in_min_x2 = sort_adj(adj_im_in, 2, False)
im_out_max_x1 = sort_adj(adj_im_out, 0, True)
im_in_max_x1 = sort_adj(adj_im_in, 0, True)
im_out_min_x2 = sort_adj(adj_im_out, 2, False)

def check_re(u, v):
    return rectangles[u][2] <= rectangles[v][0]
def check_im(u, v):
    return rectangles[u][3] <= rectangles[v][1]

edges_re_add = set()
edges_im_add = set()

# --- Step 3: Solving the Gaps ---
for i in range(n):
    if not re_out_min_y2[i]:
        continue
    j = re_out_min_y2[i][0]
    if not im_in_min_x2[j]:
        continue
    k = im_in_min_x2[j][0]
    if not re_in_max_y1[k]:
        continue
    l = re_in_max_y1[k][0]
    if not im_out_max_x1[l] or im_out_max_x1[l][0] != i:
        continue

    nodes = [i, j, k, l]
    if len(set(nodes)) == 4:
        cnt = sum(
            1 for u in nodes for v in nodes
            if u != v and (v in adj_re_out[u] or v in adj_im_out[u])
        )

```

```

    )
    if cnt == 4:
        if check_re(i, k):
            edges_re_add.add((i, k))
        if check_re(l, j):
            edges_re_add.add((l, j))
        if check_im(l, j):
            edges_im_add.add((l, j))
        if check_im(k, i):
            edges_im_add.add((k, i))

for i in range(n):
    if not im_in_max_x1[i]:
        continue
    l = im_in_max_x1[i][0]
    if not re_out_max_y1[l]:
        continue
    k = re_out_max_y1[l][0]
    if not im_out_min_x2[k]:
        continue
    j = im_out_min_x2[k][0]
    if not re_in_min_y2[j] or re_in_min_y2[j][0] != i:
        continue

    nodes = [i, j, k, l]
    if len(set(nodes)) == 4:
        cnt = sum(
            1 for u in nodes for v in nodes
            if u != v and (v in adj_re_out[u] or v in adj_im_out[u])
        )
        if cnt == 4:
            if check_re(i, k):
                edges_re_add.add((i, k))
            if check_re(l, j):
                edges_re_add.add((l, j))
            if check_im(l, j):
                edges_im_add.add((l, j))
            if check_im(k, i):
                edges_im_add.add((k, i))

Gre_edges.update(edges_re_add)
Gim_edges.update(edges_im_add)
edges_re_add.clear(); edges_im_add.clear()

adj_re_out = [[] for _ in range(n)]
adj_re_in = [[] for _ in range(n)]
adj_im_out = [[] for _ in range(n)]
adj_im_in = [[] for _ in range(n)]

for u, v in Gre_edges:
    adj_re_out[u].append(v)
    adj_re_in[v].append(u)
for u, v in Gim_edges:

```

```

adj_im_out[u].append(v)
adj_im_in[v].append(u)

# --- Step 4: Adding the Around-The-Corner edges ---
def top1(lst, sort_key, rev):
    s = sorted(lst, key=sort_key, reverse=rev)
    return s[0] if s else None

def find_cand(i, neighs, over_fn, sort_key, rev, opp_neighs):
    valid = [v for v in neighs[i] if over_fn(i, v)]
    valid.sort(key=sort_key, reverse=rev)
    if not valid:
        return None
    j2 = valid[0]
    opp2 = opp_neighs[j2]
    if top1(opp2, sort_key, rev) == i:
        return j2
    if len(valid) > 1:
        return valid[1]
    return None

for i in range(n):
    # Corner 1: Bottom-Right
    j = find_cand(
        i, adj_im_in,
        lambda u, v: rectangles[v][2] <= rectangles[u][2],
        lambda x: rectangles[x][2], True, adj_im_out
    )
    if j is not None:
        for t in adj_re_out[i]:
            if rectangles[t][1] >= rectangles[i][1]:
                edges_re_add.add((j, t))
    k = find_cand(
        i, adj_re_out,
        lambda u, v: rectangles[v][1] >= rectangles[u][1],
        lambda x: rectangles[x][1], False, adj_re_in
    )
    if k is not None:
        for t in adj_im_in[i]:
            if rectangles[t][2] <= rectangles[i][2]:
                edges_im_add.add((t, k))

    # Corner 2: Top-Right
    j = find_cand(
        i, adj_im_out,
        lambda u, v: rectangles[v][2] <= rectangles[u][2],
        lambda x: rectangles[x][2], True, adj_im_in
    )
    if j is not None:
        for t in adj_re_out[i]:
            if rectangles[t][3] <= rectangles[i][3]:
                edges_re_add.add((j, t))
    k = find_cand(

```

```

        i, adj_re_out,
        lambda u, v: rectangles[v][3] <= rectangles[u][3],
        lambda x: rectangles[x][3], True, adj_re_in
    )
    if k is not None:
        for t in adj_im_out[i]:
            if rectangles[t][2] <= rectangles[i][2]:
                edges_im_add.add((k, t))

    # Corner 3: Bottom-Left
    j = find_cand(
        i, adj_im_in,
        lambda u, v: rectangles[v][0] >= rectangles[u][0],
        lambda x: rectangles[x][0], False, adj_im_out
    )
    if j is not None:
        for t in adj_re_in[i]:
            if rectangles[t][1] >= rectangles[i][1]:
                edges_re_add.add((t, j))
    k = find_cand(
        i, adj_re_in,
        lambda u, v: rectangles[v][1] >= rectangles[u][1],
        lambda x: rectangles[x][1], False, adj_re_out
    )
    if k is not None:
        for t in adj_im_in[i]:
            if rectangles[t][0] >= rectangles[i][0]:
                edges_im_add.add((t, k))

    # Corner 4: Top-Left
    j = find_cand(
        i, adj_im_out,
        lambda u, v: rectangles[v][0] >= rectangles[u][0],
        lambda x: rectangles[x][0], False, adj_im_in
    )
    if j is not None:
        for t in adj_re_in[i]:
            if rectangles[t][3] <= rectangles[i][3]:
                edges_re_add.add((t, j))
    k = find_cand(
        i, adj_re_in,
        lambda u, v: rectangles[v][3] <= rectangles[u][3],
        lambda x: rectangles[x][3], True, adj_re_out
    )
    if k is not None:
        for t in adj_im_out[i]:
            if rectangles[t][0] >= rectangles[i][0]:
                edges_im_add.add((k, t))

    Gre_edges.update(edges_re_add)
    Gim_edges.update(edges_im_add)

    return list(Gre_edges), list(Gim_edges)

```

```

if __name__ == "__main__":
    n = int(input('Number of rectangles: '))
    print('Rectangles coordinates:')
    rectangles = [tuple(map(int, input().split())) for _ in range(n)]

    Gre, Gim = solve_rectangles(n, rectangles)

    print(f"\nGre has {len(Gre)} edges:")
    for u, v in Gre:
        print(u, v)

    print(f"\nGim has {len(Gim)} edges:")
    for u, v in Gim:
        print(u, v)

# # TESTCASES

# # Test 1: around-the-corner edges
# n1 = 7
# rectangles1 = [(0, 2, 5, 7), (0, 0, 1, 1), (2, 0, 3, 1),
#                (4, 0, 5, 1), (6, 0, 7, 3), (6, 4, 7, 5), (6, 6, 7, 7)]
# # Gre = [(1, 2), (0, 4), (3, 4), (0, 6), (2, 3), (0, 5), (3, 6), (3, 5)]
# # Gim = [(1, 5), (2, 0), (3, 0), (4, 5), (5, 6), (1, 0), (2, 5), (3, 5)]

# # Test 2: gap
# n2 = 4
# rectangles2 = [(0, 0, 1, 3), (2, 0, 5, 1), (0, 4, 3, 5), (4, 2, 5, 5)]
# # Gre = [(0, 1), (0, 3), (2, 3)]
# # Gim = [(0, 2), (1, 2), (1, 3)]

# # Test 3: sparse rectangles
# n3 = 4
# rectangles3 = [(0, 0, 1, 1), (1, 4, 2, 5), (2, 1, 3, 2), (3, 3, 4, 4)]
# # Gre = [(0, 1), (1, 2), (2, 3)]
# # Gim = [(0, 1), (2, 3), (2, 1)]

# # Test 4: many rectangles
# n4 = 8
# rectangles4 = [(0, 5, 7, 6), (0, 3, 1, 4), (2, 3, 3, 4), (4, 0, 5, 4),
#                (6, 0, 9, 1), (8, 2, 9, 9), (8, 10, 9, 11), (2, 7, 4, 9)]
# # Gre = [(1, 2), (3, 4), (0, 6), (2, 3), (7, 6), (0, 5), (3, 6), (7, 5), (3, 5)]
# # Gim = [(0, 7), (4, 0), (2, 0), (3, 0), (0, 6), (4, 5), (5, 6), (1, 0)]

# # Test 5: gap & around-the-corner edges
# n5 = 6
# rectangles5 = [(0, 0, 1, 1), (2, 0, 5, 3), (0, 2, 1, 5),
#                (4, 4, 5, 7), (0, 6, 3, 9), (4, 8, 5, 9)]
# # Gre = [(0, 1), (2, 3), (4, 5), (2, 5), (2, 1), (4, 3), (0, 3)]
# # Gim = [(0, 2), (2, 4), (1, 4), (2, 5), (0, 3), (1, 3), (3, 5)]

```